

Profiling MPI codes with Intel VTune

Trung Nguyen

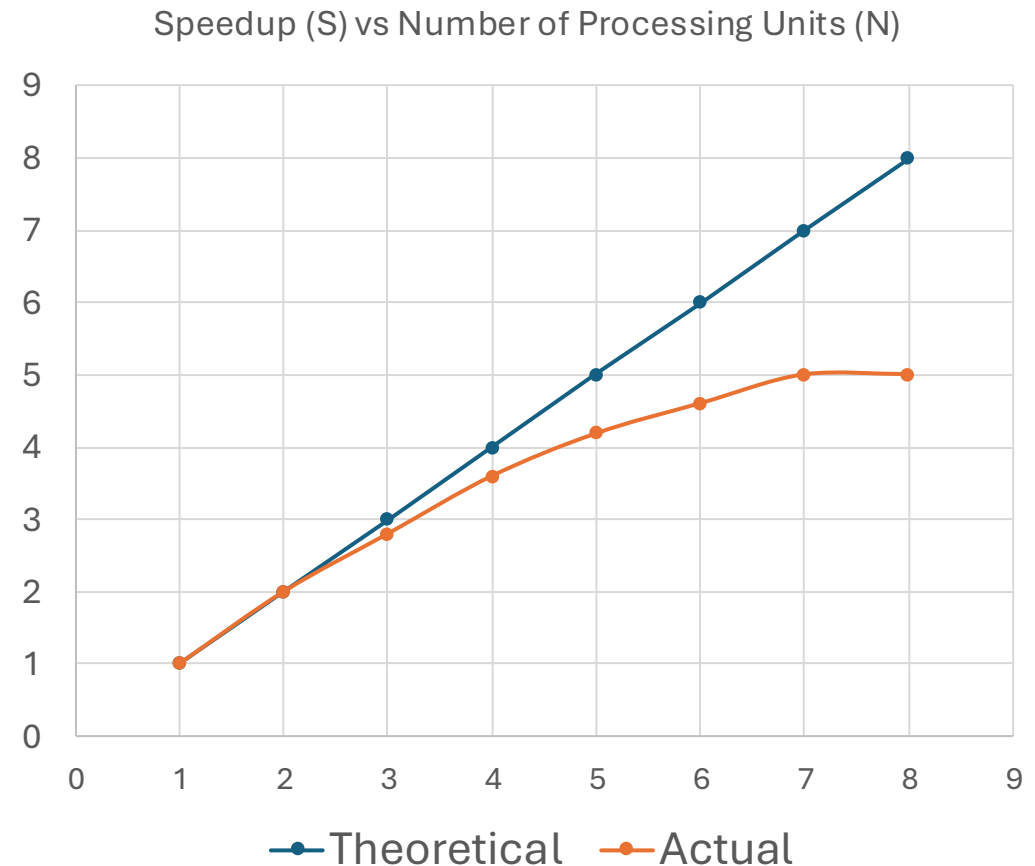
```
git clone https://github.com/rcc-uchicago/Midway3\_profiling\_knowledge\_transfer.git  
cd vtune
```

Understanding MPI codes

- MPI = Message Passing Interface
- The application built with an MPI library (e.g. OpenMPI, MPICH or Intel MPI)
- The application is launched by mpirun (or srun) in multiple processes: these processes communicate (sending and receiving data) via the dynamically linked MPI library
 - e.g. atom coords and forces between neighboring domains; pixel colors between neighboring cells
- Parallel performance: Computation gain (P/N) vs Communication overhead ($O(N)$, $O(N\log(N))$), where P = total problem size and N = number of MPI processes

Parallel efficiency: Strong scaling curve

- Parallelization introduces overhead:
 - communication/sync between workers
- Strong scaling analysis shows time to solution, or speedup, as a function of the number of MPI processes
 - linear scaling: $S_{\text{linear}} = t_1/t_N = N$
 - computation vs. communication break-even point
 - N^* where speedup stops increasing with N



What to look for with profiling an MPI code?

- **High-level:** Hotspots in the source code, functions that consume most of the calculation time
 - computation within each MPI process
 - communication
- **Lower-level:**
 - across processes: load (im)balance, collective calls
 - per-process: memory access (cache hits/misses, DRAM bandwidth, memory latency)

Hotspots are dependent on many factors: problem nature, solvers, parallelization scheme, and problem size

Building your MPI codes as usual

- `module load mpich/4.1.2+gcc-12.2.0`
- `mpicxx -g -O2 -lm -o my_mpi_app my_source_code.cpp`



store debugging info in the binary for function names and source code lines

or

```
cmake ../ -DBUILD_TYPE=RelWithDebInfo
```

```
make -j4
```

Running the applications with mpirun and vtune in a batch job or an interactive job

- sinteractive --ntasks-per-node=8
- ulimit -l unlimited

Without VTune

- mpirun -np 8 your_mpi_app input.txt

With VTune

- mpirun -np 8 /software/intel/oneapi_hpc_2024.2/vtune/latest/bin64/vtune [-collect hotspots](#) -r output_dir your_mpi_app input.txt

Two examples

- mandelbrot (see vtune/README.md in the GitLab repo)
- LAMMPS: optimizing for a new pair style (atom force calculation), in collaboration with Dr. Siva Dasetty (Ferguson's group)

First hint: The software timing breakdown without using any profiler

MPI task timing breakdown:

Section		min time		avg time		max time		%varavg		%total
Pair		223.57		223.57		223.58		0.0		99.48
Bond		9.83e-05		0.00011684		0.00014944		0.0		0.00
Neigh		0.91222		0.91774		0.92286		0.4		0.41
Comm		0.15603		0.15731		0.15901		0.2		0.07
Output		0.00045878		0.00046637		0.00049361		0.0		0.00
Modify		0.046506		0.048421		0.049106		0.4		0.02
Other				0.0444						0.02

Analyze the profiling report: Hotspots

15.9% ← using 8 procs out of 48 CPU cores

- vtune-gui ./output_dir

The screenshot shows the Intel VTune Profiler interface. The top bar includes the title 'Intel VTune Profiler' and a close button. Below it, a breadcrumb trail shows 'Welcome' and 'output-baseline-linear.midway3-0002.rcc.local'. The main navigation bar has tabs for 'Hotspots', 'Analysis Configuration', 'Collection Log', 'Summary', 'Bottom-up', 'Caller/Callee', 'Top-down Tree', 'Flame Graph', and 'Platform'. The 'Hotspots' tab is active, showing a summary of the analysis. On the left, under 'Elapsed Time', the total time is 233.219s, with CPU time at 1784.200s. Below this, the 'Top Hotspots' section lists the most active functions. A red box highlights the top three functions: 'LAMMPS_NS::PairTableUCGChirality::compute' (56.9% CPU time), 'tanhf32x' (31.0% CPU time), and 'LAMMPS_NS::PairTableUCGChirality::compute_proximity_function_der' (8.0% CPU time). On the right, the 'Hotspots Insights' section provides guidance on how to analyze the results, and the 'Explore Additional Insights' section lists other metrics like Parallelism (15.9%), Microarchitecture Usage (17.5%), and Vectorization (10.8%).

Intel VTune Profiler

Welcome × output-baseline-linear.midway3-0002.rcc.local ×

Hotspots ⓘ ⓘ

Analysis Configuration Collection Log Summary Bottom-up Caller/Callee Top-down Tree Flame Graph Platform

Elapsed Time ⓘ: 233.219s >

CPU Time ⓘ: 1784.200s

Total Thread Count: 16

Paused Time ⓘ: 0s

Top Hotspots >

This section lists the most active functions in your application. Optimizing these hotspot functions typically results in improving overall application performance.

Function	Module	CPU Time ⓘ	% of CPU Time ⓘ
LAMMPS_NS::PairTableUCGChirality::compute	Imp	1014.877s	56.9%
tanhf32x	libm.so.6	553.230s	31.0%
LAMMPS_NS::PairTableUCGChirality::compute_proximity_function_der	Imp	142.932s	8.0%
LAMMPS_NS::Pair..ev_tally_full	Imp	28.399s	1.6%
PMPI_Send	libmpi.so.12	26.279s	1.5%
[Others]	N/A*	18.482s	1.0%

*N/A is applied to non-summable metrics.

Hotspots Insights

If you see significant hotspots in the Top Hotspots list, switch to the [Bottom-up](#) view for in-depth analysis per function. Otherwise, use the [Caller/Callee](#) or the [Flame Graph](#) view to track critical paths for these hotspots.

Explore Additional Insights

Parallelism ⓘ : 15.9% 📈

Use [Threading](#) to explore more opportunities to increase parallelism in your application.

Microarchitecture Usage ⓘ : 17.5% 📈

Use [Microarchitecture Exploration](#) to explore how efficiently your application runs on the used hardware.

Vectorization ⓘ : 10.8% 📈

Use [HPC Performance Characterization](#) to learn more on vectorization efficiency of your application. A significant fraction of floating point arithmetic instructions are scalar. Use [Intel Advisor](#) to see possible reasons why the code was not vectorized.

INSIGHTS

Welcome x output-baseline-linear.midway3-0002.rcc.local x

Hotspots

Analysis Configuration Collection Log Summary **Bottom-up** Caller/Callee Top-down Tree Flame Graph Platform

Grouping: Function / Call Stack

Function / Call Stack	CPU Time
▼ LAMMPS_NS::PairTableUCGChirality::compute	1014.877s
↳ LAMMPS_NS::Verlet::run ← LAMMPS_NS::Run::command ← LAMMPS_NS::Input::execute_command ← LAMMPS_NS::PairTableUCGChirality::compute	1012.728s
↳ LAMMPS_NS::Verlet::setup ← LAMMPS_NS::Run::command ← LAMMPS_NS::Input::execute_command ← LAMMPS_NS::PairTableUCGChirality::compute	2.149s
▼ tanhf32x	553.230s
↳ LAMMPS_NS::PairTableUCGChirality::compute_proximity_function_der ← LAMMPS_NS::PairTableUCGChirality::compute	553.230s
↳ LAMMPS_NS::Verlet::run ← LAMMPS_NS::Run::command ← LAMMPS_NS::Input::execute_command ← LAMMPS_NS::PairTableUCGChirality::compute	552.070s
↳ LAMMPS_NS::Verlet::setup ← LAMMPS_NS::Run::command ← LAMMPS_NS::Input::execute_command ← LAMMPS_NS::PairTableUCGChirality::compute	1.160s
▼ LAMMPS_NS::PairTableUCGChirality::compute_proximity_function_der	142.932s
▼ LAMMPS_NS::PairTableUCGChirality::compute	142.932s
↳ LAMMPS_NS::Verlet::run ← LAMMPS_NS::Run::command ← LAMMPS_NS::Input::execute_command ← LAMMPS_NS::PairTableUCGChirality::compute	142.652s
↳ LAMMPS_NS::Verlet::setup ← LAMMPS_NS::Run::command ← LAMMPS_NS::Input::execute_command ← LAMMPS_NS::PairTableUCGChirality::compute	0.281s
▶ LAMMPS_NS::Pair::ev_tally_full	28.399s
▶ PMPI_Send	26.279s
▶ LAMMPS_NS::NPairBin<(int)0, (int)1, (int)0, (int)0, (int)0>::build	7.470s
▶ func@0x40a360	3.981s
▶ PMPI_Allreduce	1.130s
▶ PMPI_Init	0.640s

Call Stacks

< 56.5% (80.740s of 142.932s) >

Imp | LAMMPS_NS::PairTableUCGChirality::compute_proximity_functi...

Imp | LAMMPS_NS::PairTableUCGChirality::compute+0x63a900 - pair...

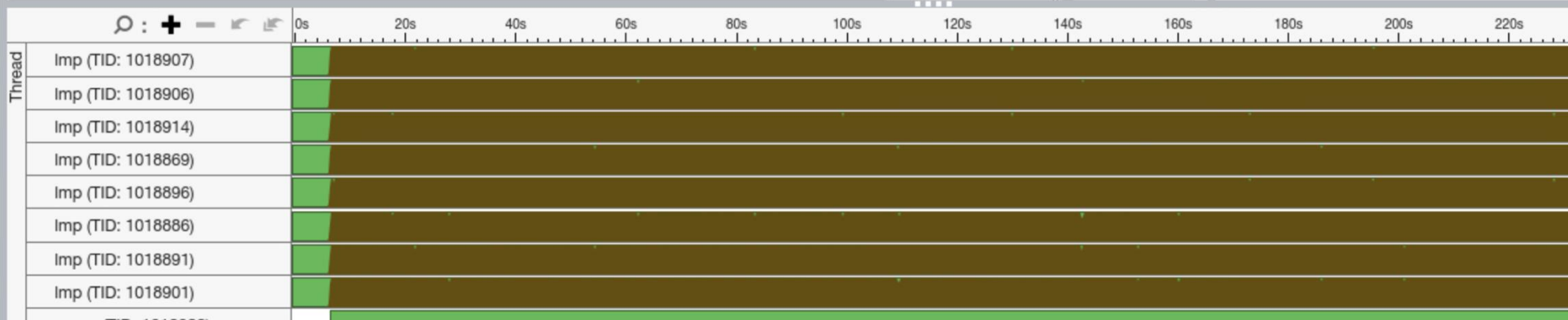
Imp | LAMMPS_NS::Verlet::run+0x21b - verlet.cpp:316

Imp | LAMMPS_NS::Run::command+0x15a0cc - run.cpp:176

Imp | LAMMPS_NS::Input::execute_command+0x76bec - input.cpp:868

Imp | LAMMPS_NS::Input::file+0x772db - input.cpp:313

Imp | main+0x6c887 - main.cpp:78



- ☒ Thread
- ☒ Running
 - ☒ CPU Time
 - ☒ Spin and Overhead...
 - ☐ CPU Sample
- ☒ CPU Utilization
- ☒ CPU Time
 - ☒ Spin and Overhead...

Hotspots highlighted in the source code (-g)

Intel VTune Profiler

Project Navi... +

test1

- r000ps
- r001ps
- r002hs
- my_result_dir.beagl...
- my_result_dir2.bea...
- output_dir2.midway...
- output_dir3.midway...
- output_dir4
- output-baseline-lin...
- output-baseline.mi...
- output-baseline.mi...
- output-hpc
- output-hpc-bad
- output-mem
- output-mpi.midway...
- r000hs
- r001hpc
- r002hpc
- r003hs
- r004hs
- r005hs
- r006hs
- ...

Welcome x output-baseline-linear.midway3-0002.rcc.local x

Hotspots

< Collection Log Summary Bottom-up Caller/Callee Top-down Tree Flame Graph Platform pair_table_ucg_chirality.cpp x

Source Assembly

Source Line ▲	Source	CPU Time: Total	CPU Time: Self
198			
199	// Compute local chirality scaled proximity function (tanh functions)		
200	double PairTableUCGChirality::compute_proximity_function(int type, double		
201	{		
202	double tanh_factor = tanh((distance - threshold_radii[type]) / (0.1 * th		
203	return neg_chirality * 0.5 * (1.0 - tanh_factor);		
204	}		
205			
206	double PairTableUCGChirality::compute_proximity_function_der(int type, dou		
207	{		
208	> double tanh_factor = tanh((distance - threshold_radii[type]) / (0.1 * th	18.2%	0s
209	return neg_chirality * 0.5 * (1.0 - tanh_factor * tanh_factor) / (0.1 *		
210	}		
211			
212	/* -----		
213			
214	void PairTableUCGChirality::compute(int eflag, int vflag)		
215	{		
216	int i,j,ii,jj,inum,jnum,itype,jtype,itable;		
217	int itype_actual, jtype_actual;		
218	double xtmp,ymtp,ztmp,dex,dely,dely,evdwl,fpair;		
219	double neg_factor is fraction value a b;		

Project Navi... +  

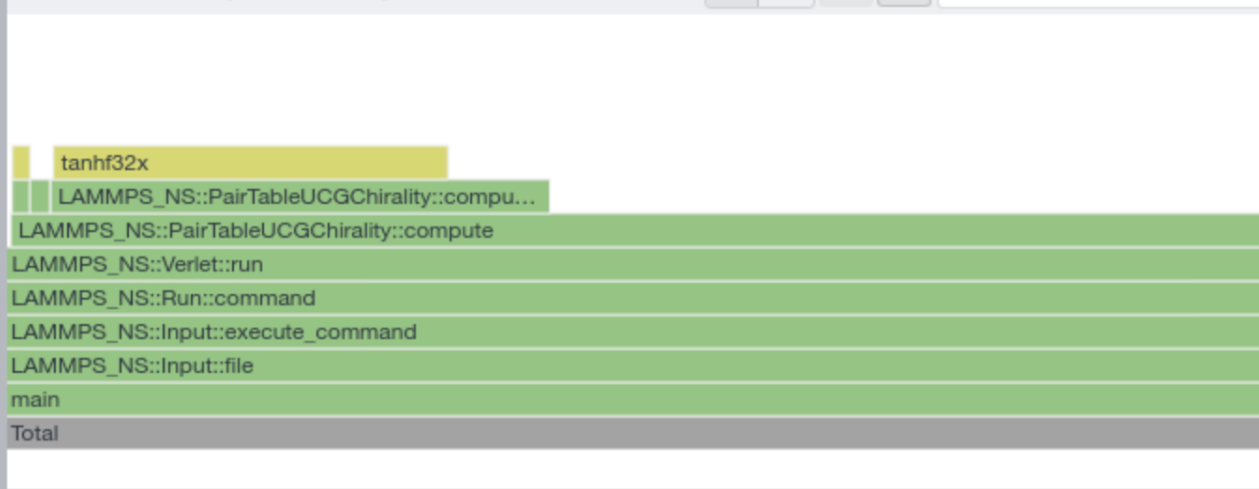
Welcome x output-baseline-linear.midway3-0002.rcc.local x

Hotspots

< Collection Log Summary Bottom-up Caller/Callee Top-down Tree

Flame Graph

Platform pair_table_ucg_chirality.cpp x

☒ User ☒ System ☒ Synchr... ☒ Other

Call Stacks

< 56.8% (1012.728s of 1784.200s) >

Imp ! LAMMPS_NS::PairTableUCGChirality::comp...

Imp ! LAMMPS_NS::Verlet::run+0x21b - verlet.cpp...

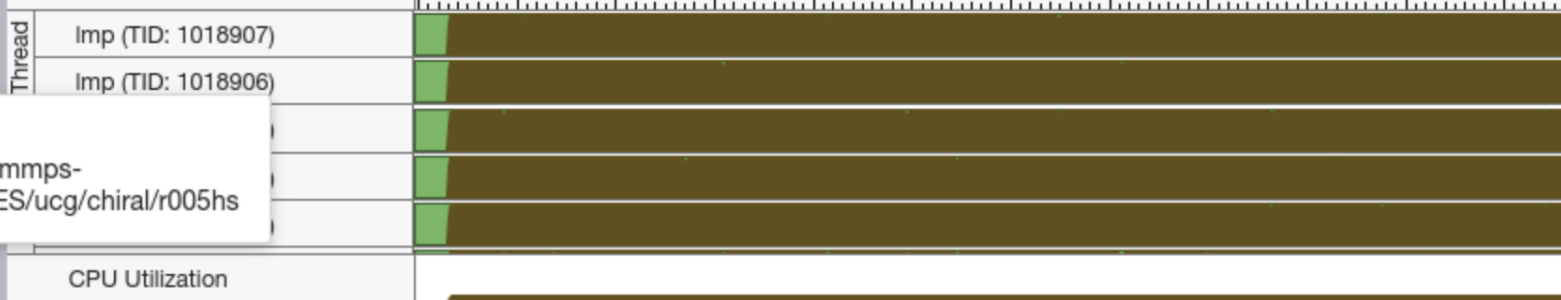
Imp ! LAMMPS_NS::Run::command+0x15a0cc - r...

Imp ! LAMMPS_NS::Input::execute_command+0x...

Imp ! LAMMPS_NS::Input::file+0x772db - input.cp...

Imp ! main+0x6c887 - main.cpp:78

0s 20s 40s 60s 80s 100s 120s 140s 160s 180s 200s 220s



CPU Utilization

FILTER 

100.0%

Any Process

Any Thread

Any Module

Any Utiliz

User functions +

Functions onl

Show inline fu

r005hs

/project/rcc/users/trung/lammps-
gitlab/examples/PACKAGES/ucg/chiral/r005hs

r005hs

r006hs

r007hs

Optimizing codes step-by-step

